浙江大学

工程前沿技术研究讲座

课 程 报 告

题目 深度生成模型综述

姓　　名 陶 雨

学　　号 22132148

年级专业 电子信息 2021 级

所在学院 控制科学与工程学院

提交日期 2021 年 11 月 9 日

深度生成模型综述

陶雨 22123148 控制科学与工程学院

摘要 深度生成模型能够生成与真实样本具有相同概率的样本，提取数据特征，使用无标记的数据进行无监督学习，有利于增强。深度生成模型经历了玻尔兹曼机（BM）、受限玻尔兹曼机（RBM）、深度信念网络（DBN）、变分自编码器（VAE）、生成对抗网络（GAN）的发展过程。GAN 由于采用对抗博弈的思想以及没有引入决定性偏置，表现出良好的数据生成能力，而被广泛研究。GAN 有 CGAN、DCGAN、LSGAN、WGAN、BGAN、SGAN、CTGAN 等多种改进型。本文总结深度生成模型的特点，回顾深度生成模型的发展过程，分析 GAN 的不同类型。

关键词 生成模型，生成对抗网络，变分自编码器，玻尔兹曼机，受限玻尔兹曼机，深度信念网络

1 引言

生成模型能接收若干样本作为训练集，生成与其概率分布相同的新样本[1,2]，正如图 1 所描述的。

图 1 生成模型的图形化描述

生成模型不需要数据标签，进行无监督学习，生成符合同样本具有相同概率分布的新数据，能够适用于需要产生近似真实样本的任务，比如，低分辨率图片生成高分辨率图片、二维图像转化为三维图像[3]、给定特定条件生成对应的样本[4]；生成模型根据从数据层到隐含层的编码过程，可以提取数据特征[5]，从而能减小其他模型训练时的输入数据维度，或者能够用于表征数据的性质；生成模型可以用部分样本无标记的数据进行训练，实现无监督学习，降低获取数据的难度；生成模型有利于实现数据增强的目标，在数据缺少的情况，比如医疗领域[6]或者小样本[7]情形，生成模型能够生成与真实数据几乎一致的数据，能够处理某一维度缺乏特定取值的数据缺失问题，使得数据更加充实。

2 发展过程

2.1 玻尔兹曼机

1983 年 Fahlman[8]提出玻尔兹曼机（Boltzmann Machines, BM）。BM 与霍普菲尔网络（Hopfield Network, HN）类似，它们两者的不同点在于：HN 中的每一个神经元与其他神经元连接，BM 虽然神经元之间互相连接但是区分为可见神经元和隐神经元，其结构示意图为图 2，其中v_1~v_4为可见神经元，h_1~h_3为隐藏神经元。

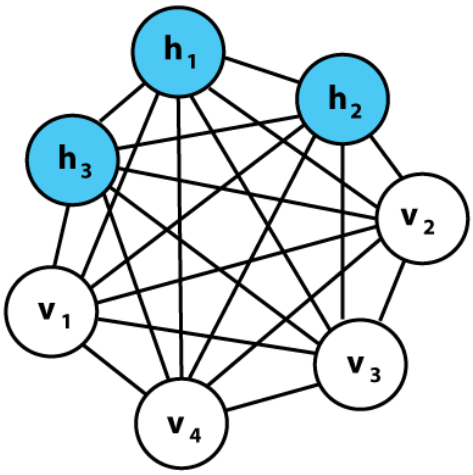

图 2 玻尔兹曼机结构示意图

玻尔兹曼机有以下特点：所有变量之间相互影响，每个变量状态以一定的概率受到其他变量的影响；每个随机变量都是二值的；包含着可观测变量和隐变量。

对于玻尔兹曼机中的随机变量 **X**，其概率分布由波尔兹曼分布得到，即

$$p(X)=\frac{1}{Z}\exp(\frac{-E(x)}{T})$$

其中 Z 为配分函数，T 表示温度，能量函数E(x)的定义为:

$$E(x)=-(\sum_{i<j}w_{ij}x_ix_j+\sum_i b_ix_i)$$

其中w_{ij}是两个变量x_i和x_j之间的权重，x_i表示状态，取值为 0 或 1，b_i表示变量x_i的偏置。

不难发现，当能量越低时，状态对应的概率越大。假设有两个变量 a 和 b 的取值都为 1，当连接权重为正时，相当于对整个网络的稳定性起到了一个积极的作用；若为负，就表示这两个变量同时取 1 不大可能，因为此时相当于给整个网络达到问题拖了后腿。

2.2 受限玻尔兹曼机

1986 年 Smolensky[9]提出了受限玻尔兹曼机（Restricted Boltzmann Machine, RBM）。相

比与 BM 的神经元之间的全连接，RBM 将网络分为可见层和隐藏层，层与层之间的神经元互相连接，但是每层内部之间的神经元之间不会互相连接，RBM 的结构与前馈神经网络（Feed Forward Neural Networks, FFNN）类似，RBM 的结构示意图为图 3。

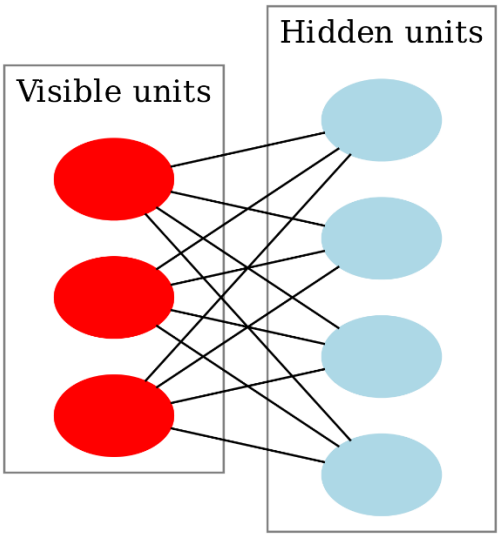

图 3　受限玻尔兹曼机结构示意图

假设x_1是从 RBM 的训练分布中的采样，ϵ为学习率，W 为权值矩阵，b 是可见层的偏置，c 是隐藏层的偏置。RBM 的训练过程为：

首先，对于所有隐藏层的节点 i，计算：

$$Q(h_{1i}=1|x_1)=sigm(c_i+\sum_j W_{ij}x_{1j})$$

根据获得的 Q 值，进行采样：

$$sample\ h_{1i}\in\{0,1\}\ from\ Q(h_{1i}|x_i)$$

然后，对于所有的可见层节点 j，计算：

$$P(x_{2j}=1|h_1)=sigm(b_j+\sum_i W_{ij}h_{1i})$$

根据获得的 P 值，进行采样：

$$sample\ x_{2j}\in\{0,1\}\ from\ P(x_{2j}=1|h_1)$$

借着，对于所有隐藏层的节点 i，计算：

$$Q(h_{2i}=1|x_2)=sigm(c_i+\sum_j W_{ij}x_{2j})$$

最后更新权重和偏置，然后如此循环往复。

$$
\begin{aligned}
&W \leftarrow W + \epsilon\left(\mathbf{h}_1 \mathrm{x}_1' - Q\left(\mathbf{h}_{2.} = 1 \mid \mathrm{x}_2\right)\mathrm{x}_2'\right) \\
&\mathbf{b} \leftarrow \mathbf{b} + \epsilon\left(\mathrm{x}_1 - \mathrm{x}_2\right) \\
&\mathbf{c} \leftarrow \mathbf{c} + \epsilon\left(\mathbf{h}_1 - Q\left(\mathbf{h}_{2.} = 1 \mid \mathrm{x}_2\right)\right)
\end{aligned}
$$

2.3 深度信念网络

2006 年 Hinton[10]提出了深度信念网络（Deep Belief Network, DBN）。DBN 由多个受限玻尔兹曼机堆积而成，其结构示意图为图 4。

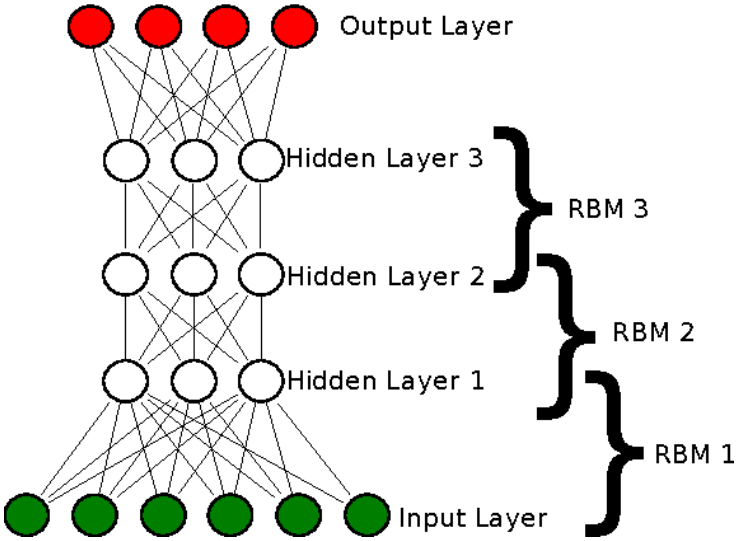

图 4　深度信念网络的结构示意图

DBN 的参数学习过程分为无监督贪婪逐层训练和 Wake-Sleep 调优两个阶段。在第一阶段，首先充分训练第一个 RBM，固定第一个 RBM 的权重和偏置，将其隐层神经元的状态作为第二个 RBM 的输入向量，再对第二个 RBM 进行训练，如此依次进行。如果顶层的 RBM 除了有显性神经元还包括代表分类标签的神经元，则需要将其一起进行训练。第二个阶段是参数的调优过程，分为 Wake（认知过程）和 Sleep（生成过程）两个子阶段。在 Wake 阶段，外界的输入特征在向上权重的作用下产生出各层的输出，再基于梯度下降法来修改各层的下行权重；在 Sleep 阶段通过顶层表示和向下权重生成底层的状态，同时修改向上的权重。DBN 在特征提取过程中，先将输入信号拉成向量再投入到网络中，忽略了数据中存在的空间结构，在处理图像和视频这样的二维或高维信号时会存在一定问题．同时，当输入信号的维数过高以及网络的深度过深时，网络中需要训练的参数很多，这给存储和计算提了很高的要求。

2.4 变分自编码器

2013 年 Kingma[11]提出变分自编码器（Variational AutoEncoder, VAE）。VAE 由编码器

（Encoder）和解码器（Decoder）两部分构成，Encoder 负责将真实样本编码成以每个样本为条件的正态分布的均值与方差的组合序列，Decoder 负责将编码得到的正态分布数据转换成生成样本，以实现生成模型的数据。VAE 的结构示意图为图 5。

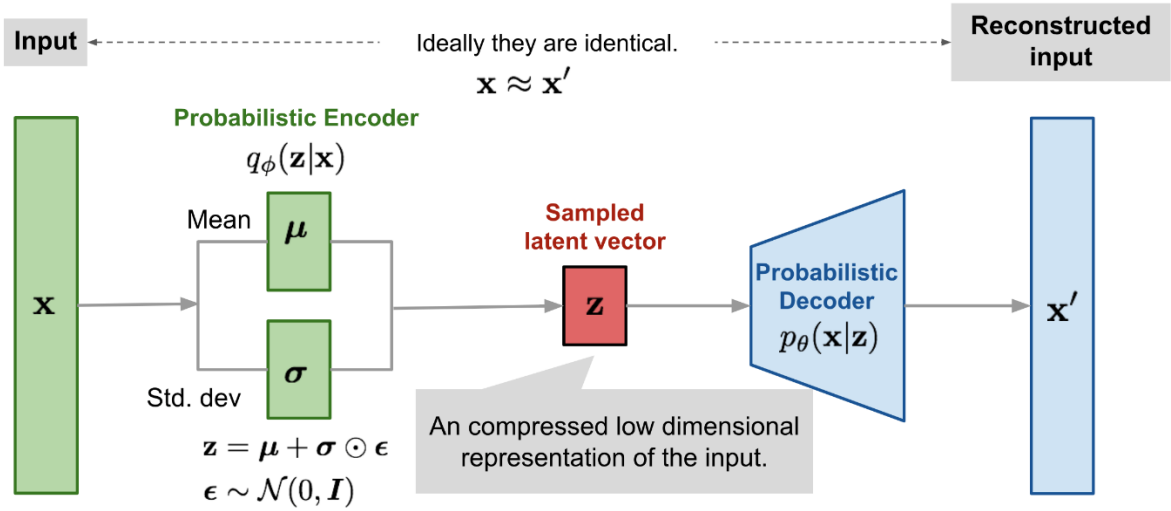

图 5　变分自编码器结构示意图

VAE 训练时的输出误差设定为真实样本与生成样本的重构误差。VAE 为每个样本构造专属的正态分布，然后通过从每个专属正态分布中采样 Z 来重构数据。VAE 让所有的p(Z|X)趋向于标准正态分布，一方面防止噪声为零，另一方面使得模型具有生成能力。

$$p(Z) = \sum_X p(Z|\ X)p(X) = \sum_X \mathcal{N}(0,I)p(X) = \mathcal{N}(0,I)\sum_X p(X) = \mathcal{N}(0,I)$$

根据上式，当p(Z|X)趋向于标准正态分布，p(Z)也为标准正态分布，从而满足 VAE 的先验假设，这样模型训练完毕后我们可以通过从标准正态分布中的采样值来重构样本，而不是从每个单独的正态分布中的采样值来重构样本。

为了满足p(Z|X)趋向于标准正态分布的要求，VAE 在重构误差基础上加上 KL 散度作为正则项来约束，KL 散度定义为：

$$\mathcal{L}_{\mu,\sigma^2} = \frac{1}{2}\sum_{i=1}^{d}\left(\mu_{(i)}^2 + \sigma_{(i)}^2 - \log\sigma_{(i)}^2 - 1\right)$$

VAE 其实在自编码器（AutoEncoder, AE）基础上，对 Encoder 的结果加上了高斯噪声，使得 Decoder 对噪声具有鲁棒性；而 KL 散度正则项是为了使 Encoder 的结果是零均值的。当 Decoder 的重构误差大于 KL 散度值时，VAE 会降低噪声，使得生成样本更加容易，对应于 KL 散度值增加，重构误差减小；当 Decoder 的重构误差小于 KL 散度值时，VAE 会增加噪声，提升生成样本的难度，以便提升生成样本的能力，对应于 KL 散度值减小，重

构误差增大。

2.5 对抗生成网络

2014 年，Goodfellow[12]提出了生成对抗网络（Generative Adversarial Network, GAN）。GAN 没有假设单个损失函数，而是让判别器 D 和生成器 G 之间进行零和博弈。其结构示意图为图 6。

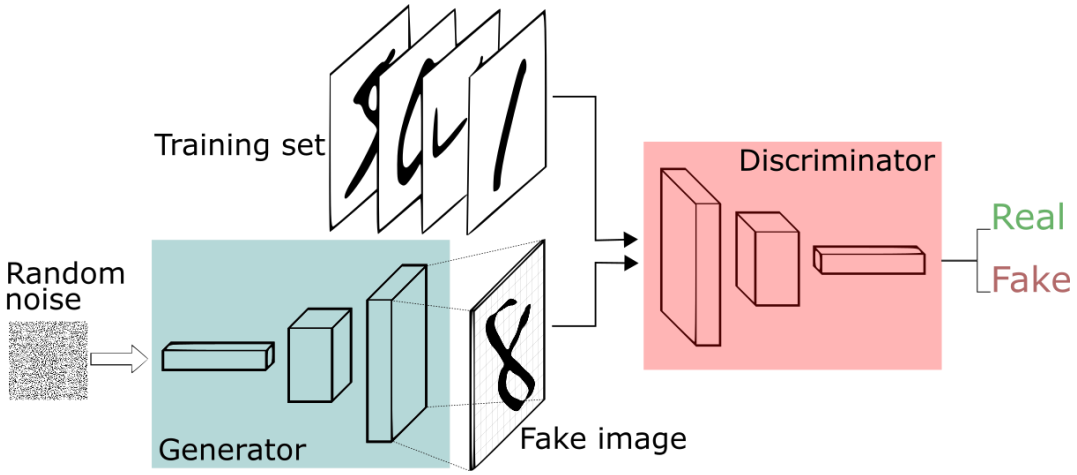

图 6　对抗生成网络的结构示意图

一方面，生成器 G 要以生成假样本为目的，以欺骗判别器 D 误以为真实样本，其损失函数可以定义为：

$$L_G^{GAN} = E[\log(D(G(z)))]$$

另外一方面，判别器 D 要以区分真实样本 x 和假样本 G(z)为目的，其损失函数可以定义为：

$$L_D^{GAN} = E[\log(D(x))] + E[\log(1 - D(G(z)))]$$

在 GAN 训练过程中，首先我们对判别器 D 进行训练，输入为真实样本与虚假样本，输出为样本的真实性 1 或者 0，目的是让 D 能辨别出真实样本与虚假样本；然后，我们固定判别器 D 中的系统参数，将生成器 G 与之前训练完的判别器联合进行训练，训练过程中生成器 G 中的参数进行实时地更新，判别器 D 参数保持不变，输入为噪声 z，输出为样本的真实性 1 或 0，目的是为了让判别器 D 对噪声 z 生成的虚假样本 G(z)判断为真实的；最后，重复上述两个步骤，直至两者达到纳什平衡，使得生成器 G 和判别器 D 都有很好的性能，之后我们可以单独抽出生成模型来用于生成数据。

VAE 的损失函数为：

$$L_{\theta,\phi}^{VAE} = -\mathbb{E}_{q_\phi(z|x_i)}\left[\log\left(p\left(x_i|\ g_\theta(z)\right)\right)\right] + KL\left(q_\phi\left(z|\ x_i\right)\|\ p(z)\right)$$

与 VAE 相比，GAN 采用模型对抗的思想，通过最优化两个损失函数的方法，让生成模型能够达到很好的训练效果，其生成数据的效果会更强；此外，GAN 的判别器相当于一个样本的度量模型，而这是 VAE 所不具备的。基于以上考虑，本文接下来讨论 GAN 的不同优化类型。

3 对抗生成网络类型

3.1 CGAN

GAN 只能生成随机的接近真实的样本，但无法生成特定的样本。基于以上考虑，Mirza[4] 提出了条件生成对抗网络（Conditional Generative Adversarial Network, CGAN）。CGAN 的结构示意图为图 7。

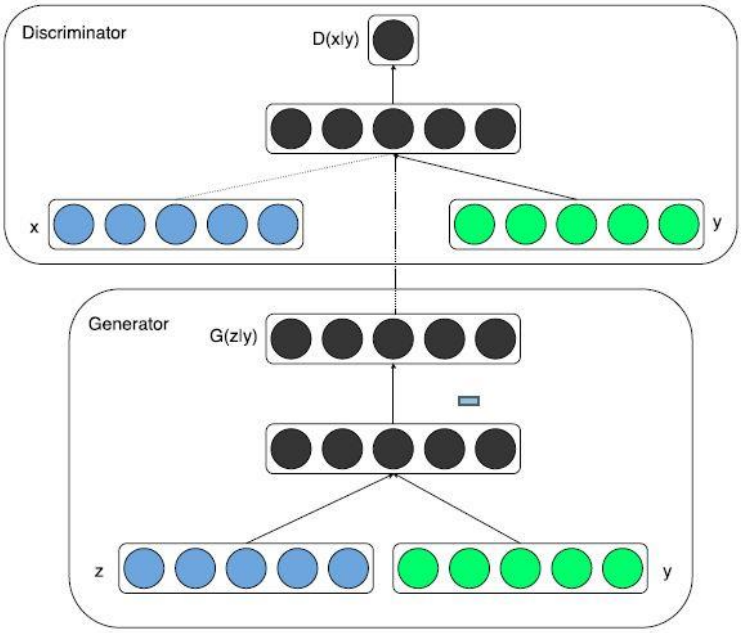

图 7　CGAN 结构示意图

判别器 D 和生成器 G 的损失函数为：

$$L_D^{CGAN} = E[\log(D(x,y))] + E[\log(1 - D(G(z),y))]$$
$$L_G^{CGAN} = E[\log(D(G(z),y))]$$

结构示意图与损失函数显示，判别器 D 和生成器 G 的输入中都增添了条件信息 y，这就表明：判别器 D 的目标变为判断样本是否满足条件 y 的要求，或者是否是标签 y 对应的样本；生成器 G 的目标变为生成满足条件 y 或者与标签 y 对应的虚假样本。训练过程中多增加一维输入信息，能够使得模型具有更强的生成能力。

3.2 DCGAN

GAN 应用在图像领域时，网络的训练参数过多，使得神经网络结构变得复杂；另外一方面，CNN 应用在图像领域时，往往采用的是监督学习的方式。为了解决以上问题，Radford[13]提出了深度卷积生成对抗网络（Deep Convolutional Generative Adversarial Network, DCGAN）。DCGAN 的结构示意图为图 8。

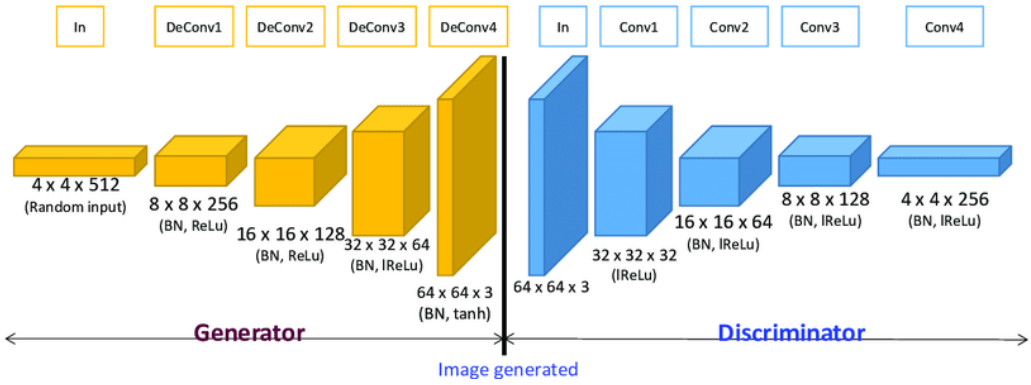

图 8　DCGAN 结构示意图

DCGAN 并没有直接更改损失函数或者对 GAN 的原理进行分析，而是直接对网络结构施加限制从而实现更为强大的生成模型。DCGAN 使用指定步长的卷积层代替卷积层；生成器 G 和判别器 D 都使用 BN 层（Batch Normalization）；移除全连接层；生成器除去输出层采用 Tanh 外，全部使用 ReLU 作为激活函数；判别器所有层都使用 LeakyReLU 作为激活函数。

DDGAN 将卷积网络引入到 GAN 中，减小了训练过程中的网络参数，降低了网络的复杂度，从而解决了 GAN 难以训练的问题，但由于没有直接对 GAN 的原理层面进行修正，所以仍然具有 GAN 所拥有的缺点。

3.3 LSGAN

在训练过程中，GAN 的损失函数会导致梯度消失问题，为了解决这样的问题，Mao[14]提出了最小二乘生成对抗网络（Least Square Generative Adversarial Network, LSGAN）。

LSGAN 修改 GAN 的损失函数的表达形式，将交叉熵损失函数转变为最小二乘损失函数形式，具体为下式。

$$L_D^{LSGAN} = E\left[(D(x)-1)^2\right] + E\left[D(G(z))^2\right]$$
$$L_G^{LSGAN} = E\left[(D(G(z))-1)^2\right]$$

LSGAN 能够最小化 Pearson 散度，更加严格地惩罚离群数据，能够生成比 GAN 更加

清晰的图像；最小二乘保证离群惩罚更大，使得 LSGAN 在学习参数过程中比 GAN 更加稳定。LSGAN 与 GAN 的损失函数比较如图 9 所示。

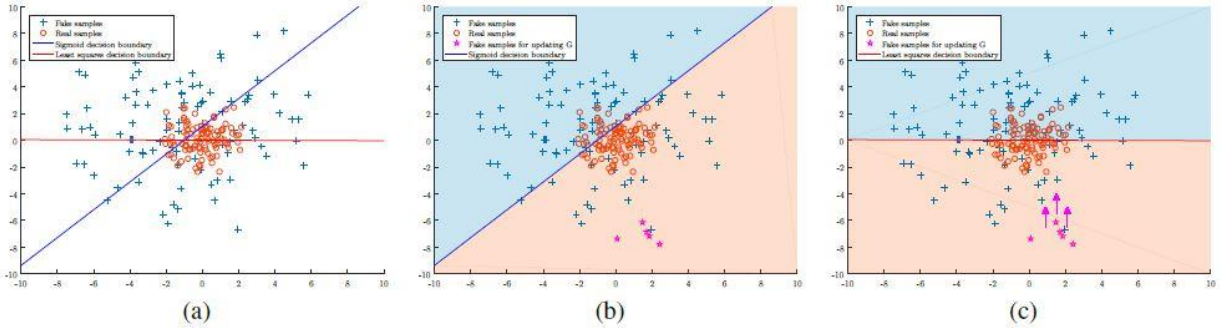

图 9　LSGAN 与 GAN 的损失函数比较图

LSGAN 的缺点是，对离离群点的过度惩罚，可能导致样本生成的”多样性”降低，生成样本很可能只是对真实样本的简单模仿和细微改动。

3.4 WGAN

Arjovsky[15]提出了 Wasserstein 生成对抗网络（Wasserstein Generative Adversarial Network, WGAN）。WGAN 的损失函数表达式为下式。

$$L_D^{WGAN} = E[D(x)] - E[D(G(z))]$$
$$L_G^{WGAN} = E[D(G(Z))]$$
$$W_D \leftarrow \mathrm{clip_b y_v alue}\left(\mathrm{W_D}, -0.01, 0.01\right)$$

WGAN 通过剪裁 D 网络的方式，对 D 网络进行稳定更新，能够有效防止梯度消失的问题。WGAN 与 GAN 采用评价准则区别如图 10 所示。

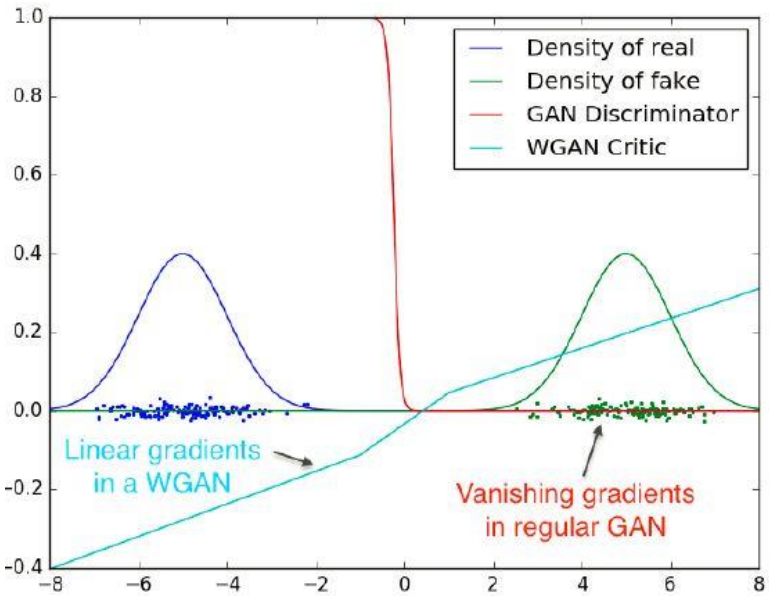

图 10　WGAN 与 GAN 的评价准则区别图

但是，WGAN 一味地通过裁剪权重因素来保证训练稳定性，可能导致生成低质量的数据或者图片。

3.5 BGAN

Hjelm[16]提出边界寻找生成对抗网络（Boundary-seeking Generative Adversarial Network, BGAN）。BGAN 生成器以不断生成决策边界上的样本为目标，其损失函数表达式为下式。

$$L_D^{BEGAN} = D_{AE}(x) - k_t D_{AE}(G(z))$$
$$L_G^{BEGAN} = D_{AE}(G(Z))$$
$$k_{t+1} = k_t + \lambda\left(\gamma D_{AE}(x) - D_{AE}(G(z))\right)$$

GAN 依赖于完全可微的生成样本，因此不能够生成离散数据。BGAN 引入一种离散数据训练 GAN 的方法，利用判别器的估计误差来计算生成样本的重要性权重，从而为训练生成器提供策略梯度，重要性权重与判别器的决策边界有很强的联系，而 BGAN 能够搜索出该边界，从而实现对离散数据的生成。此外，BGAN 边界目标寻找可以扩展到连续数据，能够提升训练的稳定性。

3.6 SGAN

Huang[17]提出了堆叠生成对抗网络（Stacked Generative Adversarial Network, SGAN）。其结构示意图为图 11。

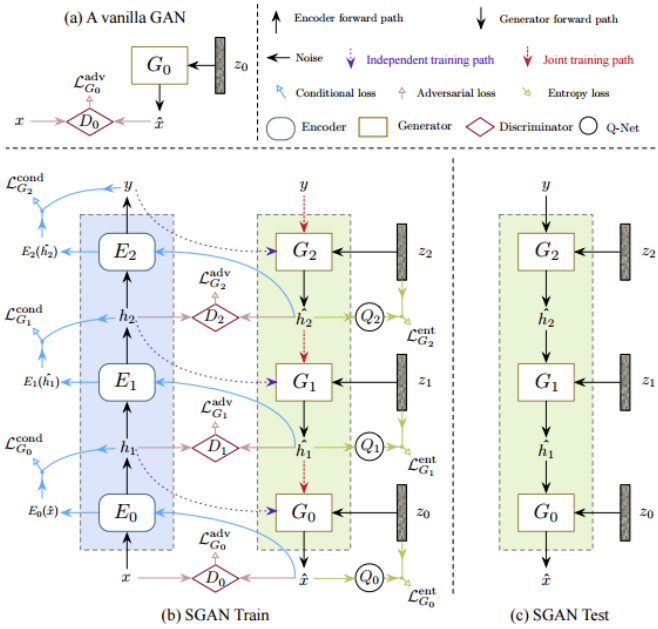

图 11 SGAN 结构示意图

SGAN 由自下而上的 GAN 堆积而成，每个 GAN 都学会以更高级别的特征来表示较低

级别的特征。SGAN 在每个特征层次引入一个表征判别器，来使得生成器的表征与自下而上的判别网络的表征保持一致。SGAN 首先独立训练每层的 GAN，然后再端到端地训练整个模型。相比 GAN，SGAN 能够生成更加清晰的数据。

3.7 CTGAN

Xu[18]提出了条件表格生成对抗网络（Conditional Tabular Generative Adversarial Network, CTGAN）。其结构示意图为图 12。

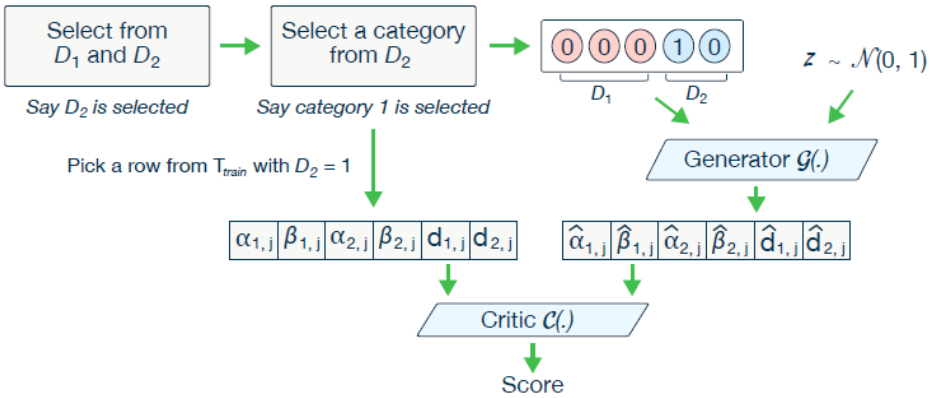

图 12　CTGAN 结构示意图

生成器 G 的网络结构为：

$$
\begin{aligned}
&h_0 = z \oplus cond \\
&h_1 = h_0 \oplus \mathrm{ReLU}\left(\mathrm{BN}\left(\mathrm{FC}_{|\text{ cond }|+|z|\to 256}\left(h_0\right)\right)\right) \\
&h_2 = h_1 \oplus \mathrm{ReLU}\left(\mathrm{BN}\left(\mathrm{FC}_{|\text{ cond }|+|z|+256\to 256}\left(h_1\right)\right)\right) \\
&\hat{\alpha}_i = \tanh\left(\mathrm{FC}_{|\text{ cond }|+|z|+512\to 1}\left(h_2\right)\right) \qquad 1 \le i \le N_c \\
&\hat{\beta}_i = \text{ gumbel }_{0.2}\left(\mathrm{FC}_{|\text{ cond }|+|z|+512\to m_i}\left(h_2\right)\right) \qquad 1 \le i \le N_c \\
&\mathbf{d}_i = \text{ gumbe }1_{0.2}\left(\mathrm{FC}_{|\text{cond}|+|z|+512\to |D_i|}\left(h_2\right)\right) \qquad 1 \le i \le N_d
\end{aligned}
$$

鉴别器 D 的网络结构为：

$$
\begin{cases}
h_0 = \mathbf{r}_1 \oplus \ldots \oplus \mathrm{r}_{10} \oplus \mathrm{cond}_1 \oplus \ldots \oplus \mathrm{cond}_{10} \\
h_1 = \mathrm{drop}\left(\mathrm{leaky}_{0.2}\left(\mathrm{FC}_{10|\mathbf{r}|+10|\text{ cond }|\to 256}\left(h_0\right)\right)\right) \\
h_2 = \mathrm{drop}\left(\mathrm{leaky}_{0.2}\left(\mathrm{FC}_{256\to 256}\left(h_1\right)\right)\right) \\
\mathcal{C}(\cdot) = \mathrm{FC}_{256\to 1}\left(h_2\right)
\end{cases}
$$

GAN 生成表格数据时会出现以下问题：难以处理同时包含连续数据和离散数据的混合数据；难以处理非高斯分布的数据；难以处理多模态分布的数据；难以处理列取值不平衡

性的数据。CTGAN 采用特定模态归一化、条件生成器以及采样训练的方法，能够有效解决上述问题，生成符合特定条件的表格数据。

参考文献

[1] SALAKHUTDINOV R. Learning Deep Generative Models [J]. 2015, 2(1): 361-385.

[2] GOODFELLOW I, BENGIO Y, COURVILLE A. Deep Learning [M]. MIT press, 2016.

[3] GOODFELLOW I. Nips 2016 Tutorial: Generative Adversarial Networks [J]. arXiv preprint arXiv:00160, 2016.

[4] MIRZA M, OSINDERO S J A P A. Conditional Generative Adversarial Nets [J]. 2014.

[5] HE K, ZHANG X, REN S, et al. Deep Residual Learning for Image Recognition: Proceedings of the IEEE conference on computer vision and pattern recognition, 2016 [C].

[6] BERMUDEZ C, PLASSARD A J, DAVIS L T, et al. Learning Implicit Brain Mri Manifolds with Deep Learning: Medical Imaging 2018: Image Processing, 2018 [C]. International Society for Optics and Photonics.

[7] BARTUNOV S, VETROV D. Few-Shot Generative Modelling with Generative Matching Networks: International Conference on Artificial Intelligence and Statistics, 2018 [C]. PMLR.

[8] FAHLMAN S E, HINTON G E, SEJNOWSKI T J. Massively Parallel Architectures for Al: Netl, Thistle, and Boltzmann Machines: National Conference on Artificial Intelligence, AAAI, 1983 [C].

[9] SMOLENSKY P. Information Processing in Dynamical Systems: Foundations of Harmony Theory [R]: Colorado Univ at Boulder Dept of Computer Science, 1986.

[10] HINTON G E, OSINDERO S, TEH Y-W J N C. A Fast Learning Algorithm for Deep Belief Nets [J]. 2006, 18(7): 1527-1554.

[11] KINGMA D P, WELLING M J A P A. Auto-Encoding Variational Bayes [J]. 2013.

[12] GOODFELLOW I, POUGET-ABADIE J, MIRZA M, et al. Generative Adversarial Nets [J]. 2014, 27.

[13] RADFORD A, METZ L, CHINTALA S J A P A. Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks [J]. 2015.

[14] MAO X, LI Q, XIE H, et al. Least Squares Generative Adversarial Networks: Proceedings of the IEEE international conference on computer vision, 2017 [C].

[15] ARJOVSKY M, CHINTALA S, BOTTOU L. Wasserstein Generative Adversarial Networks: International conference on machine learning, 2017 [C]. PMLR.

[16] HJELM R D, JACOB A P, CHE T, et al. Boundary-Seeking Generative Adversarial Networks [J]. 2017.

[17] HUANG X, LI Y, POURSAEED O, et al. Stacked Generative Adversarial Networks: CVPR, 2017 [C].

[18] XU L, SKOULARIDOU M, CUESTA-INFANTE A, et al. Modeling Tabular Data Using Conditional Gan [J/OL] 2019, arXiv:1907.00503.